

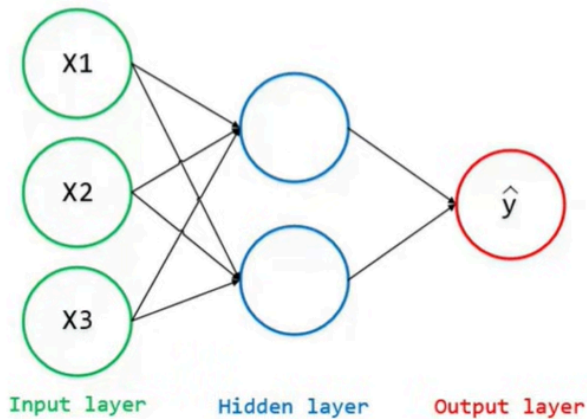
Normalization

What is Normalization

05 June 2024 10:32

Normalization in deep learning refers to the process of transforming data or model outputs to have specific statistical properties, typically a mean of zero and a variance of one.

What do we normalize?

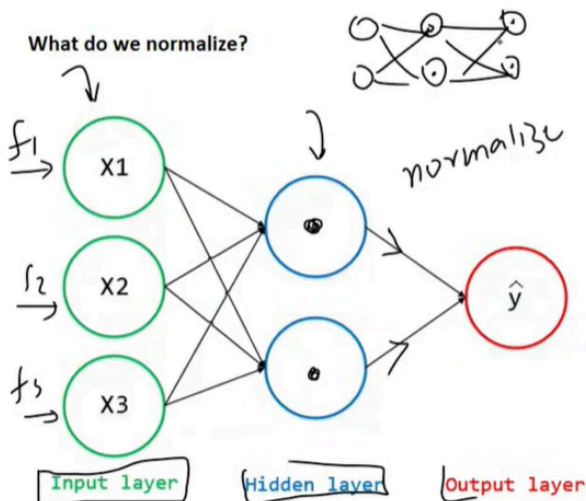


min-max

$$\frac{x_i - \mu}{\sigma}$$

$\mu = 0$
 $\sigma = 1$

What do we normalize?



min-max

$$\frac{x_i - \mu}{\sigma}$$

$\mu = 0$
 $\sigma = 1$

f_1	f_2	f_3
-	-	-
-	-	-
-	-	-

Benefits of Normalization in Deep Learning

- Improved Training Stability:
 - Normalization helps to stabilize and accelerate the training process by reducing the likelihood of extreme values that can cause gradients to explode or vanish.

NORMALIZATION IN MACHINE LEARNING AND TRANSFORMERS

1. INTRODUCTION TO NORMALIZATION

Normalization is a preprocessing technique used in machine learning to scale input features into a specific range or distribution. The goal is to make training more stable, faster, and more efficient by ensuring that features contribute proportionally to the learning process.

In many datasets, features have different scales. For example, one feature may range from 0 to 1, while another may range from 0 to 10000. Without normalization, features with larger numeric ranges can dominate gradient updates, leading to unstable or inefficient learning.

2. WHY NORMALIZATION IS USED

Normalization is used for the following reasons:

1. Faster convergence: Gradient descent converges faster when features are on similar scales.
2. Stable training: Prevents numerical instability during optimization.
3. Better performance: Helps models generalize better by reducing bias toward large-scale features.
4. Improved gradient flow: Reduces issues like exploding or vanishing gradients in deep networks.

Common feature normalization techniques include:

- Min-Max scaling
- Z-score standardization (mean = 0, variance = 1)

3. INTERNAL COVARIATE SHIFT

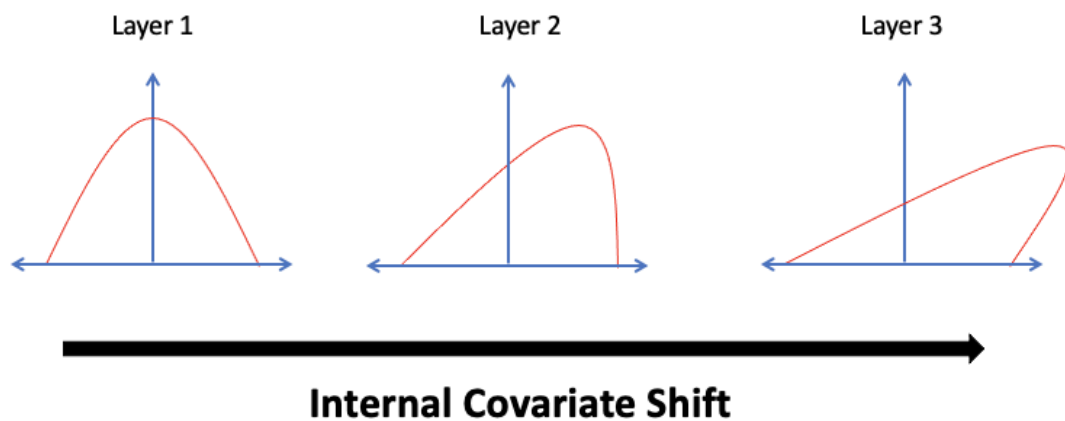
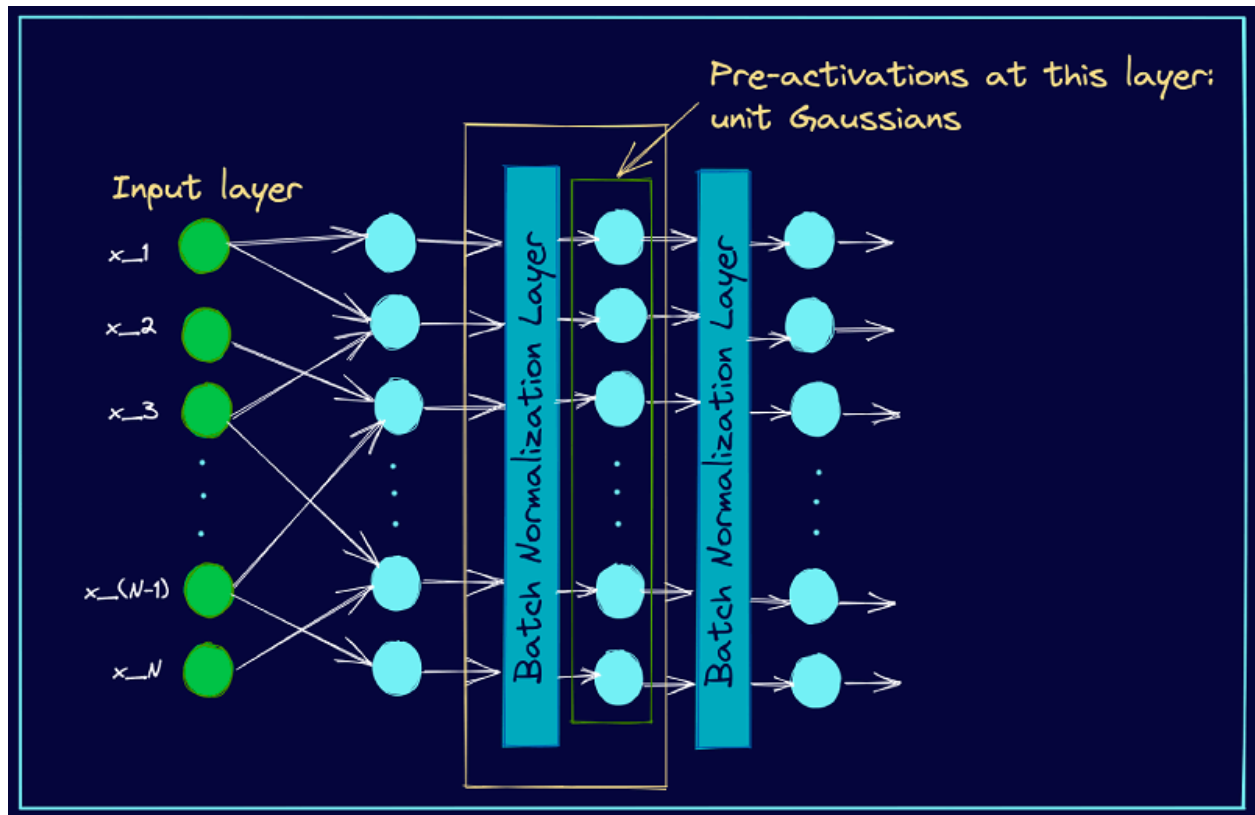
Internal Covariate Shift refers to the change in the distribution of network activations during training as parameters of previous layers change.

In deep neural networks, each layer receives inputs from the previous layer. As weights are updated, the distribution of these inputs keeps changing. This forces each layer to continuously adapt to new input distributions, slowing down training.

Problems caused by internal covariate shift:

- Slower convergence
- Need for lower learning rates
- Sensitivity to initialization
- Training instability in deep networks

Batch Normalization was introduced to reduce this effect.



4. BATCH NORMALIZATION

Batch Normalization (BatchNorm) normalizes the inputs of a layer using statistics computed over a mini-batch.

For a given feature activation x :

Step 1: Compute batch mean

$$\mu_B = (1/m) \sum x_i$$

Step 2: Compute batch variance

$$\sigma_B^2 = (1/m) \sum (x_i - \mu_B)^2$$

Step 3: Normalize

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$$

Step 4: Scale and shift

$$y_i = \gamma \hat{x}_i + \beta$$

where:

- γ and β are learnable parameters
- ϵ is a small constant for numerical stability

Advantages of Batch Normalization:

- Reduces internal covariate shift
- Allows higher learning rates
- Acts as a regularizer
- Improves convergence speed

Limitations:

- Depends on batch size
- Not suitable for sequential or variable-length tasks in some cases
- Training and inference behavior differ due to batch statistics

5. LAYER NORMALIZATION

Layer Normalization (LayerNorm) is a normalization technique applied across the features of a single training example rather than across a batch.

Instead of computing statistics across the batch dimension, LayerNorm computes mean and variance across the feature dimension for each individual sample.

For a hidden vector x of dimension d :

Step 1: Compute mean

$$\mu = (1/d) \sum x_i$$

Step 2: Compute variance

$$\sigma^2 = (1/d) \sum (x_i - \mu)^2$$

Step 3: Normalize

$$\hat{x}_i = (x_i - \mu) / \sqrt{\sigma^2 + \epsilon}$$

Step 4: Scale and shift

$$y_i = \gamma \hat{x}_i + \beta$$

Key differences from BatchNorm:

- Works on each sample independently
- Does not depend on batch size
- Same behavior during training and inference

6. WHY LAYER NORMALIZATION IS USED IN TRANSFORMERS

Transformers process sequences and rely heavily on stable representations across layers.

LayerNorm is preferred over BatchNorm because:

1. Independence from batch size: Transformers often use variable or small batch sizes.
2. Sequence modeling compatibility: Each token representation is normalized independently.
3. Stability in deep architectures: Helps stabilize residual connections.
4. Consistent behavior during training and inference.

In Transformer architecture, Layer Normalization is typically applied:

- After attention sublayers
- After feed-forward networks
- In some architectures, before sublayers (Pre-LN Transformers)

7. SUMMARY

- Normalization improves training stability and convergence in machine learning.
- Internal Covariate Shift describes changing activation distributions during training.
- Batch Normalization reduces this issue using batch statistics but depends on batch size.
- Layer Normalization normalizes per sample across features and is independent of batch size.
- Transformers use Layer Normalization because it is more stable and suitable for sequential processing.

Aspect	BatchNorm	LayerNorm
Normalization Direction	Across mini-batches for each feature	Across features for each sample
Batch Size Dependence	Requires a sufficiently large batch size	Works with any batch size
Computation Scope	Per-feature across samples	Per-sample across features
Inference Behavior	Uses moving averages for mean/variance	Same as training behavior
Use Cases	CNNs, feedforward networks	RNNs, Transformers